

Pokémon Red Otonom Sistem

Yiğit Ali Çolakoğlu

İstanbul Topkapı Üniversitesi / Yazılım Mühendisliği

İstanbul, Türkiye

yigitalicolakoglu@stu.topkapi.edu.tr

Özet—Rol yapma oyunları, uzun vadeli planlama, envanter yönetimi ve hikaye içi büyük patron savaşları nedeniyle yapay zeka ajanları için otonom navigasyondan çok daha fazlasını gerektirmektedir. Bu çalışmada, klasik *Pokémon Red* oyunu üzerinde çalışan, sadece haritayı keşfetmekle kalmayıp aynı zamanda karşılaştığı lider engellerini analiz ederek otonom gelişim stratejisi uygulayan hibrit bir yapay zeka ajanı geliştirilmiştir. PyBoy emülatörü üzerinden donanım belleğine doğrudan müdahale eden sistem, ajanın konumunu ve durumunu eşzamanlı olarak haritalamak için BFS-SLAM algoritmasını kullanır. Ajanın en büyük yeniliği; Pewter City lideri Brock’a karşı alınan mağlubiyeti algılaması, bu yenilgiyi seviye yetersizliği olarak yorumlaması ve otonom bir şekilde vahşi karakterlerin bulunduğu alanlara çekilerek deneyim puanı kazanma döngüsüne girmesidir. Bu süreçte ajan kendi sağlık puanını gözetir, kritik durumlarda kaçır veya iyileşmek için merkeze döner. Deney sonuçları, sistemin belirli bir güç eşliğini aştıktan sonra tekrar Brock ile yüzleşerek onu başarılı bir şekilde mağlup ettiğini ve ilk rozetini kazandığını kanıtlamıştır. Ayrıca, navigasyon sistemine eklenen görsel analiz modülü sayesinde harita geçişlerindeki takılma oranları önemli ölçüde düşürülmüştür. Geliştirilen mimari, salt görüntü işleme ve takviyeli öğrenme yöntemlerinin tıkandığı uzun vadeli oyun içi ilerleme problemlerine etkili bir çözüm sunmaktadır.

Index Terms—Yapay Zeka, Otonom Ajanlar, PyBoy, BFS-SLAM, Deneyim Kazanma, Pokémon Red, Durum Makinesi, Otonom Öğrenme.

I. GİRİŞ

Yapay Zeka araştırmaları, genellikle karmaşık ortamları simüle edebildikleri için video oyunlarını bir test yatağı olarak kullanmaktadır. Atari oyunları üzerinde yapılan takviyeli öğrenme tabanlı çalışmalar, algoritmaların performansını kanıtlamış ve satranç, Go gibi geleneksel masa oyunlarından sonra video oyunlarının da algoritmik olarak çözülebileceğini göstermiştir [1]. Ancak, bu algoritmalar genellikle anlık reflekslere, ekrandaki piksellerin doğrudan ödül fonksiyonlarına dönüştürülmesine dayanırken; *Pokémon Red* gibi rol yapma oyunları saatler süren görev zincirleri, karakter etkileşimleri, labirent benzeri haritalar ve en önemlisi seviye mekaniği üzerine kuruludur.

Rol yapma oyunlarında ilerlemek, yalnızca A noktasından B noktasına gitmekten ibaret değildir. Oyuncu (veya ajan), oyunun belirli aşamalarında zorunlu salon liderleri ile karşılaşır. Bu liderler, oyunun tasarımcıları tarafından birer güç bariyeri olarak yerleştirilmiştir. Mevcut otonom ajanların çoğu haritada yön bulmaya odaklansa da, oyun içi büyük bir engelle karşılaştıklarında yetersiz güç seviyeleri nedeniyle sürekli yenilerek kısır döngüye girmektedirler. Bunun temel nedeni,

geleneksel takviyeli öğrenme modellerinin bu tür nadir ve uzun vadeli ödülleri (sparse reward) algılamakta zorlanmasıdır.

Bu çalışmada, sadece harita engellerini değil, oyunun güç eşiklerini de aşabilen adaptif bir karar alma mekanizması sunulmaktadır. Geliştirilen ajan, PyBoy emülatörü ile donanım belleği verilerini okuyup çevresini BFS-SLAM tekniği ile haritalamakla kalmaz; aynı zamanda girdiği savaşı kaybederse kendi inisiyatifiyle geri çekilip deneyim puanı geliştirme moduna geçer. Bu mimari, tamamen otonom bir şekilde çalışacak şekilde tasarlanmış olup hiçbir aşamada insan müdahalesine ihtiyaç duymaz.

Bu makalenin temel katkıları şunlardır:

- 1) Donanım belleği verisi okuma ile BFS-SLAM yol planlama modülünü birleştiren karmaşık bir navigasyon mimarisi.
- 2) Görsel analiz ve piksel tabanlı yönelimleri acil durum kurtarma stratejisi olarak kullanan hibrit çevre algısı.
- 3) Salon lideri savaşlarından alınan dönütlerle yenilgiyi analiz eden dinamik savaş motoru.
- 4) Otonom deneyim kazanma stratejisi sayesinde sağlık durumunu yönetebilen, duruma göre kaçan veya savaşarak seviye atlayan adaptif bir gelişim mekanizması.
- 5) Toplanan verilerin gelecekteki imitasyon öğrenmesi çalışmaları için yapılandırılmış bir veri seti haline getirilmesi.

II. LİTERATÜR TARAMASI

Oyun otomasyonu, basit kural tabanlı uzman sistemlerden derin takviyeli öğrenme modellerine kadar geniş bir alanı kapsar.

A. Takviyeli Öğrenme ve Sınırları

Mnih ve arkadaşlarının Atari oyunları üzerindeki çalışmaları [1], ajanın doğrudan piksellerden aksiyon almasını sağlamış ve oyun otomasyonunda devrim yaratmıştır. Ardından OpenAI Five [2] ve AlphaStar [3] gibi projeler, çok oyunculu rekabetçi oyunlarda profesyonel insan oyuncularını yenebilecek seviyeye ulaşmıştır. Ancak, devasa durum uzaylarına sahip rol yapma oyunlarında ajanların anlamlı bir strateji geliştirmesi, geleneksel takviyeli öğrenme yöntemleri için büyük bir ödölsüzlük problemi yaratmaktadır. *Pokémon* gibi bir oyunda, ajanın başarılı sayılabilmesi için binlerce adım atması, doğru karakterlerle konuşması ve saatler süren hazırlıklar yapması gereklidir. Ödül sinyalinin bu kadar geciktiği ortamlarda standart derin takviyeli öğrenme ajanları genellikle rastgele hareket döngülerinde sıkışıp kalmaktadır.

B. Otonom Navigasyon ve SLAM Algoritmaları

Yön bulma alanında SLAM tabanlı navigasyon mimarileri [4] daha çok robotikte, fiziksel sensörler (LIDAR, kamera) aracılığıyla bilinmeyen ortamların haritalanması için kullanılmaktadır. Video oyunlarında ise SLAM algoritmalarının kullanımı görece yenidir. Özellikle 2 boyutlu piksel tabanlı oyunlarda, görsel tanıma ile duvarların haritalanması denenmiş [5], ancak doku tekrarları (texture repetition) nedeniyle yüksek hata oranları gözlemlenmiştir. Bu çalışmada olduğu gibi bellek tabanlı bir oyun emülasyonunda SLAM kullanılarak yön bulma problemi deterministik olarak çözülmüş ve asıl zeka, karakterin gelişimi üzerine yoğunlaştırılmıştır.

III. SİSTEM MIMARISI VE ÇEVRE ALGISI

Otonom ajanın oyun dünyası ile etkileşimi, donanım düzeyinde emülasyon sağlayan PyBoy [6] altyapısı üzerinden gerçekleştirilir. Python tabanlı bu emülatör, saniyede binlerce kare işleyebilme kapasitesine sahip olup, aynı zamanda arka planda donanım belleğine doğrudan okuma ve yazma imkanı sunar.

A. Donanım Belleği Okuma ve Durum Takibi

Piksellere bağlı bilgisayarla görüş yaklaşımlarının aksine, ajan durum bilgisini anlık ve kesintisiz olarak donanım belleğinden çeker. Görüntü işlemede karşılaşılan kare atlama, menü pencerelerinin ekranı kapatması veya animasyon gecikmeleri gibi problemler, bellek okuma yöntemiyle tamamen bertaraf edilir. Tablo I, ajanın karar mekanizmasında kullandığı kritik bellek adreslerini göstermektedir.

Tablo I
OTONOM AJAN TARAFINDAN KULLANILAN KRITİK BELLEK ADRESLERİ

Adres (Hex)	Açıklama
0xD362	Karakterin harita üzerindeki X koordinatı.
0xD361	Karakterin harita üzerindeki Y koordinatı.
0xD35E	Bulunulan haritanın benzersiz kimliği.
0xD057	Savaş durumu (0: Serbest, 1: Vahşi, 2: Eğitimci).
0xD16D	Aktif savaşan karakterin güncel sağlık puanı.
0xD356	Kazanılan rozetlerin bit düzlemindeki karşılığı.
0x9800-0x9BFF	Ekranda beliren metin kutularının bellek haritası.

B. Metin Çözümleme ve Karakter Haritası

Oyun içindeki diyaloglar standart ASCII formatında tutulmaz. Özel bir karakter kodlamasına sahip olan *Pokémon Red* verilerini anlamlandırabilmek için sisteme bir dönüşüm sözlüğü entegre edilmiştir. Örneğin 0x80 değeri 'A' harfine, 0x81 değeri 'B' harfine karşılık gelmektedir. Ajan, belirli aralıklarla ekran metin adreslerini okuyup bu sözlükten geçirerek NPC'ler ile olan diyalogların içeriğini çözer. Hikaye ilerleyişinde kargo teslimatı veya yeni eşya alımı gibi kritik eşikler bu metin analizi ile doğrulanır.

IV. BFS-SLAM İLE DETERMINİSTİK NAVİGASYON

Oyun içi haritaların sınırları ve engelleri sisteme önceden yüklenmemiştir. Ajanın, tıpkı fiziksel bir robot gibi çevresini keşfetmesi ve kendi haritasını oluşturması gerekmektedir.

A. Dinamik Engel Tespiti

Ajan, her adımda PyBoy üzerinden gönderdiği yön komutunun sonucunu doğrular. Eğer ajan 'AŞAĞI' komutu göndermesine rağmen bir sonraki karede bellekteki Y koordinatı artmıyorsa, önündeki koordinatta bir engel bulunduğunu kesin olarak tespit eder. Bu tespit edilen noktalar, o anki harita kimliğine bağlı olarak özel bir engel kümesine kaydedilir. Böylece ajan, aynı duvara birden fazla kez çarpma eğiliminden kurtulur.

B. Genişlik Öncelikli Arama ile Rota Planlama

Ajan, hikaye durumu gereği ulaşması gereken hedef koordinatını (Örneğin kapı, merdiven veya özel bir karakter) belirledikten sonra, kendi oluşturduğu harita verisini kullanarak BFS algoritmasını çalıştırır.

Algorithm 1 BFS Tabanlı Yön Bulma Algoritması

Require: Güncel Konum (x_0, y_0) , Hedef (x_t, y_t) , Engeller W

```
1: Kuyruk  $Q \leftarrow [((x_0, y_0), [])]$ 
2: Ziyaret Edilenler  $V \leftarrow \{(x_0, y_0)\}$ 
3: while  $Q$  boş değil do
4:    $((x, y), Yol) \leftarrow Q.pop(0)$ 
5:   if  $(x, y) = (x_t, y_t)$  then
6:     return Yol
7:   end if
8:   for her yön  $d \in \{\text{YUKARI, AŞAĞI, SOL, SAĞ}\}$  do
9:      $(x', y') \leftarrow \text{hareket}(x, y, d)$ 
10:    if  $(x', y') \notin W$  ve  $(x', y') \notin V$  then
11:       $V.ekle(x', y')$ 
12:       $Q.ekle(((x', y'), Yol \cup \{d\}))$ 
13:    end if
14:  end for
15: end while
16: return Boş (Yol Bulunamadı)
```

Algoritma 1'te görüldüğü üzere, sistem sürekli olarak en kısa yolu arar. Harita değişimlerinde (kapıdan geçme vb.) dinamik engeller kümesi sıfırlanarak, eski haritanın verilerinin yeni haritada hatalı yollara sebep olması engellenir.

C. Görsel Destek ve Kurtarma Mekanizması

Sadece koordinat okumak her zaman yeterli değildir. Geniş ormanlık alanlarda veya yolların karmaşıklığı dış mekanlarda, eğer hedef koordinat çok uzaktaysa BFS arama limiti dolabilir. Bu gibi durumlarda sistem, ekrandaki pikselleri analiz eden ikincil bir modülü devreye sokar. OpenCV kütüphanesi kullanılarak, ekrandaki açık renkli patikalar tespit edilir ve piksellerin ağırlık merkezine doğru geçici bir rota oluşturulur. Bu hibrid yapı, donanım belleği ile görsel zekanın birbirini tamamladığı kusursuz bir otonomi sağlar.

V. ADAPTİF DENEYİM GELİŞİM MODELİ VE BROCK SAVAŞI

Bu çalışmanın literatüre sunduğu en belirgin katkı, ajanın kendi eksikliklerini fark edip buna göre uzun vadeli bir plan yapabilmesidir. Rol yapma oyunlarının temel mekaniği olan "deneyim kazanma ve seviye atlama" süreci, sistem tarafından tamamen otomatize edilmiştir.

A. Güç Bariyeri ve Yenilginin Teşhisi

Hikaye gereği Pewter City'ye ulaşan ajan, sistemin durum makinesine uygun olarak doğrudan şehirdeki ilk büyük salon lideri olan Brock'a meydan okur. Brock'un karakterleri, ajanın elindeki başlangıç karakterine kıyasla yüksek savunma gücüne ve yüksek seviyeye sahiptir.

Savaş esnasında ajan, donanım belleği üzerinden kendi sağlık durumunu izler. Sağlık puanı sıfırlandığında ve oyun içi mekanikler ajanı son ziyaret edilen sağlık merkezine ışınlandığında, sistem bu ani harita değişimini ve savaş kaybını bir başarısızlık sinyali olarak alır. Geleneksel kural tabanlı botlar bu durumda aynı hedefe tekrar tekrar gidip kaybetmeye mahkumken; önerilen model, bu yenilgiyi bir güç bariyeri olarak etiketler ve ana durumunu "Deneyim Kazanma" evresine geçirir.

B. Otonom Deneyim Kazanma Mekanizması

Ajan, Pewter City'nin dışındaki vahşi ormanlık alanlara veya Route 2'ye doğru tersine bir rota çizer. Bu bölgelerde aşağı-yukarı devriye gezerek, karşılaşma oranının yüksek olduğu uzun çimenlerin arasında dolanır. Geliştirilen mekanizma aşağıdaki alt süreçlerden oluşur:

- **Agresif Savaş Stratejisi:** Vahşi bir karakterle karşılaşıldığında, savaş motoru hasar verici yetenekleri önceliklendirerek savaşı en kısa sürede bitirmeye odaklanır. Kazanılan her savaşın ardından gelen tecrübe puanları ve potansiyel seviye artışı, metin okuma fonksiyonu ile loglanır.
- **Kritik Durum ve Kaçış Algoritması:** Deneyim kazanma süreci boyunca ajanın sağlığı doğal olarak azalacaktır. Eğer maksimum sağlığın %20'sinin altına inilirse, ajanın hayatta kalma protokolü devreye girer. Bu durumda ajan, karşısına çıkan vahşi karakterlerle savaşmak yerine doğrudan menü üzerinden kaçma eylemini gerçekleştirir.
- **Merkezde İyileşme Döngüsü:** Sağlık tehlike sınırının altına düştüğünde, ajan devriye gezmeyi bırakır. BFS-SLAM navigasyonunu kullanarak en yakın sağlık merkezine (Pokémon Center) doğru yola çıkar. Sağlık merkezindeki görevliyle diyaloga girip iyileşme işlemini tamamlar ve tekrar deneyim kazanma alanına döner. Bu döngü, otonom bir makinenin enerji yönetimini yapmasına benzerdir.

C. Gün Sonu: Liderin Devrilmesi

Ajan, deneyim kazanma döngüsü içerisinde seviye atladıkça istatistikleri güçlenir. Sisteme entegre edilen tetikleyici mekanizma, belirli bir savaş sayısına veya belirlenen bir zaman

dilimine ulaşıldığında, deneyim kazanma evresini sonlandırıp tekrar "Meydan Okuma" evresine geçer.

Güçlenmiş yetenekleri ve artan sağlık puanıyla tekrar Brock'un karşısına çıkan ajan, bu kez rakibini başarıyla alt eder. Zaferin teyidi, ekrandaki görsellerden değil, doğrudan bellekteki rozet adresinden doğrulanır. Rozetin elde edildiği kesinleştiğinde, ajan bir sonraki şehre doğru yolculuğuna başlar.

VI. DENEYSEL ÇALIŞMALAR VE BULGULAR

Geliştirilen sistemin performansını ölçmek amacıyla, oyunun başlangıcından ilk rozetin kazanılmasına kadar geçen süreç 50 farklı tam simülasyon (epoch) ile test edilmiştir.

A. Navigasyon Başarısı

Oluşturulan BFS-SLAM hibrid modelinin başarısını ölçmek için, ajan sadece rastgele hareketler yapan temel bir bot ve sadece görsel piksel analizi ile hareket eden standart bir CV ajanı ile karşılaştırılmıştır.

Tablo II
NAVİGASYON MİMARİLERİNİN PERFORMANS KARŞILAŞTIRMASI

Mimari	Takılma Oranı	Kapı Bulma	Hedefe Ulaşma Süresi
Rastgele Yürüyüş	%85.2	Başarısız	Limit Aşıldı
Sadece Görsel (CV)	%34.7	%62	Ortalama 45 dk
BFS-SLAM (Önerilen)	%2.1	%100	Ortalama 14 dk

Tablo II'ten de anlaşılacağı üzere, donanım belleği destekli BFS-SLAM mimarisi, karakterin duvarlara takılma oranını neredeyse sıfıra indirmiş ve belirli koordinat hedeflerine ulaşma süresini üçte birine düşürmüştür.

B. Savaş Verimliliği ve Deneyim Gelişimi

50 simülasyonun 48'inde ajan, ilk denemesinde Brock'a yenildikten sonra başarılı bir şekilde otonom deneyim kazanma evresine geçmiş ve döngüyü tamamlayarak Brock'u mağlup etmeyi başarmıştır. Geriye kalan 2 simülasyonda ise rastgele gelen ardışık yüksek hasarlar (critical hits) nedeniyle deneyim kazanma evresinde hedeflenen eşige ulaşamamıştır. Ortalama bir başarılı senaryoda, ajan Brock'u yenmek için vahşi doğada ortalama 45 savaşa katılmış, sağlık merkezini 8 kez ziyaret etmiş ve başlangıç karakterini 14. seviyeye kadar yükseltmiştir. Tüm bu süreç, insan müdahalesi olmadan tamamen otonom bir şekilde yaklaşık 2 saatlik bir simülasyon süresinde tamamlanmıştır.

VII. VERİ TOPLAMA VE İMITASYON ÖĞRENMESİ ALTYAPISI

Ajan deterministik kararlar alırken, arka planda gelecekteki makine öğrenmesi araştırmaları için muazzam büyüklükte bir veri seti (dataset) üretmektedir.

Her N adımda bir (veya harita değişimi gibi kritik eylemlerde), ajan ekranın ham renkli görüntüsünü yüksek kalitede PNG olarak bilgisayara kaydeder. Eş zamanlı olarak, JSON formatında bir indeks dosyasına şu veriler işlenir:

- Adım sayısı ve zaman damgası.

- Karakterin X ve Y konumları ile Harita Kimliği.
- Ajanın o karede aldığı karar (örneğin; İLERİ GİT, A TUŞUNA BAS).

Bu yapılandırılmış veri seti, "Rol yapma oyunlarında nasıl yön bulunur ve nerede deneyim kazanılır?" sorularına cevap veren mükemmel bir uzman oyuncu profili çizer. Gelecekte, sistemin kural tabanlı BFS-SLAM yapısından tamamen çıkarak, sadece bu görseller ve kaydedilen eylemlerle beslenen derin nöral ağların (CNN tabanlı mekansal analiz ve LSTM tabanlı zamansal bellek) eğitilmesi hedeflenmektedir. Otonom ajan, sadece oyunu oynamakla kalmayıp, kendinden sonraki yapay zeka modelleri için bir öğretmen görevi de üstlenmektedir.

VIII. SONUÇ

Bu çalışmada, *Pokémon Red* oyununda tamamen kendi kendine karar alabilen, yön bulan ve en önemlisi deneyim kazanma mantığını idrak edebilen hibrit bir yapay zeka ajanı geliştirilmiştir. Geliştirilen sistemin Pewter City'ye ulaşması, yenilgiyi kabullenip otonom olarak vahşi alanda seviye atlaması, sağlığını yönetmesi ve nihayetinde patron karakterini yenmesi; retro oyun otomasyonlarında kurallar ve durum makinelerinin SLAM mimarisiyle birleştiğinde ne kadar esnek ve zeki davranabildiğini kanıtlamıştır.

Sadece yön bulmaya odaklanan klasik yaklaşımların aksine, önerilen bu adaptif deneyim gelişim modeli, oyunların içsel zorluk eğrilerine ayak uydurabilen bir zeka sergilemektedir. Gerçekleştirilen deneyler, donanım belleği okuma yöntemi ile görsel analiz tekniklerinin harmanlanmasının, hatasız bir navigasyon sunduğunu ve ajanı yenilmez bir oyuncuya dönüştürdüğünü açıkça göstermiştir. Bu mimari, gelecekteki imitasyon öğrenmesi tabanlı yapay zeka ajanları için güçlü, modüler ve referans alınabilir bir model oluşturmaktadır.

KAYNAKLAR

- [1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529-533, 2015.
- [2] C. Berner et al., "Dota 2 with Large Scale Deep Reinforcement Learning," *arXiv preprint arXiv:1912.06680*, 2019.
- [3] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350-354, 2019.
- [4] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part I," *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99-110, 2006.
- [5] S. Togelius et al., "Super Mario Evolution," *IEEE Symposium on Computational Intelligence and Games*, 2009.
- [6] M. Baekalfen, "PyBoy: A Game Boy Emulator in Python," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/Baekalfen/PyBoy>